

Codebase Memory MCP

`codebase-memory-mcp` is a high-performance code intelligence engine that acts as a Model Context Protocol (MCP) server. It indexes codebases into a persistent, structural knowledge graph using tree-sitter.

Why is it useful?

Instead of forcing AI coding assistants to use token-heavy file search tools (like `grep` or reading entire files), it allows the AI to query the codebase's structure. This provides instant answers to complex architectural queries (like finding all dependencies or call chains for a function) using 99% fewer tokens.

Installation (Windows)

You only need to install this **once per computer**. The setup script installs the tool globally and configures it for your AI agents. You do not need to reinstall it for every new project.

Standard Installation:

```
Invoke-WebRequest -Uri https://raw.githubusercontent.com/DeusData/codebase-memory-mcp/main/install.ps1 -OutFile install.ps1; .\install.ps1
```

Installation with 3D Visualizer (UI variant): If you want the interactive 3D graph visualizer, use the `--ui` flag during installation:

```
Invoke-WebRequest -Uri https://raw.githubusercontent.com/DeusData/codebase-memory-mcp/main/install.ps1 -OutFile install.ps1; .\install.ps1 --ui
```

Running the 3D Visualizer

If you installed the UI variant, you can launch the visualizer at any time using:

```
codebase-memory-mcp --ui=true --port=9749
```

Then, open your web browser to <http://localhost:9749> to explore your codebase as an interactive 3D graph.

A-Z Workflow in a Project

1. **Start your project session:** Open your IDE or terminal.
2. **First connection (Automatic Indexing):** When the AI agent first connects to the MCP server, it will automatically index the project in milliseconds.
3. **Commit the Graph (Optional):** If working on a team, the agent can commit the compressed graph (`.codebase-memory/graph.db.zst`) so teammates don't have to reindex from scratch.

4. **Ask Architectural Questions:** Ask the AI complex structural questions instead of text searches. E.g.,
"Trace all inbound calls to the *createTask* function."
5. **Auto-sync:** As you write code, the background watcher will detect file changes and incrementally update the graph database.

Available MCP Tools

Once installed, the AI assistant receives access to these 14 specific tools:

1. **get_architecture:** Returns a high-level overview of languages, packages, entry points, routes, hotspots, boundaries, layers, and clusters.
2. **manage_adr:** Manages Architecture Decision Records (ADRs) to persist architectural choices across chat sessions.
3. **detect_changes:** Maps uncommitted Git changes to affected symbols with risk classification (Impact Analysis).
4. **trace_path:** Resolves function call chains (inbound or outbound) across files and packages.
5. **semantic_query:** Vector search across the graph. Powered by local Nomic embeddings. Use to find conceptually similar code without exact keyword matches.
6. **search_graph:** Structural search using regex name patterns, label filters, and min/max degree (e.g., finding all functions named **Handler* with high complexity).
7. **search_code:** Graph-augmented text search (grep) limited over indexed files.
8. **find_dead_code:** Scans the entire graph to find functions or variables with zero callers (excluding entry points).
9. **execute_cypher:** Allows raw graph database queries. E.g., `MATCH (f:Function)-[:CALLS]->(g) WHERE f.name = 'main' RETURN g.name`
10. **link_services:** Detects and maps cross-service HTTP requests (matching a frontend fetch to a backend route).
11. **detect_channels:** Detects Socket.IO, EventEmitter, and pub/sub patterns across multiple languages.
12. **get_node_details:** Retrieves comprehensive structural information about a specific file or symbol node.
13. **compare_nodes:** Uses Jaccard scoring and MinHash to find near-clone detection between functions.
14. **index_repository:** Forces a manual re-index of the repository.